

TROVE: Adaptive Agent Skill Orchestration via Trace-Grounded Route Validation and Editing

Tianxing Wang, Mingming Zhao, Shuai Huang, Huiyang Xu,
Chaoyue Niu, Shengzhong Liu, Fan Wu

Abstract

Agents tend to optimize, select, or constrain execution structures before decisive runtime outcomes are observed. However, such pre-execution commitment creates an orchestration bottleneck: when intermediate evidence invalidates the pending continuation, agents must either execute stale steps or re-plan broadly, compounding errors, wasting computation, and discarding progress. We thus propose Trace-grounded Route Orchestration via Validation and Editing (TROVE), which revises only what runtime evidence invalidates. Offline, TROVE distills evaluated workflow-search traces into atomic and composite skills and an outcome-conditioned transition graph, preserving stable fragments while exposing outcome-dependent decisions. Online, it treats a planned route as provisional: after committing one top-level skill, the controller retains a valid continuation, inserts a trace-supported local response, or replaces only the invalid suffix. Evaluations across code-generation, question-answering, and math reasoning benchmarks with different LLM backbones show that TROVE delivers a stronger quality-efficiency trade-off than existing baselines of dataset-level optimization, query-level architecture selection, and graph-constrained scheduling. Quality gains are largest when outcomes change the appropriate continuation, whereas early termination yields substantial efficiency gains on near-saturated tasks. Ablations further show that composite skills capture most offline benefits, insertion enables local correction, and suffix replacement primarily improves efficiency. These findings establish selective route editing as a general principle for adaptive agent orchestration.

1 Introduction

Agents increasingly solve complex tasks by orchestrating multiple reasoning, generation, verification, and tool-use steps. The task performance depends not only on the capability of each component, but also on when and in what order those components are invoked. A strong solver may need a critic only when its answer is uncertain; a generated program may require repair only after a failed test; and a planned verification step becomes redundant once an earlier skill has already produced a valid final answer. Agent workflow orchestration is therefore not merely a mechanism for composing model calls. It determines whether intermediate evidence is converted into useful computation or ignored by a stale execution path.

Existing work has substantially improved how agent workflows are constructed, but places most structural decisions

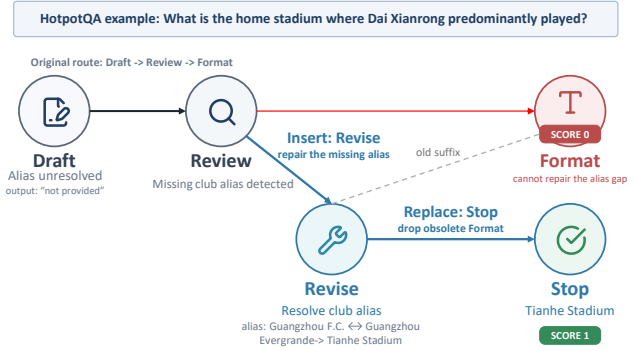


Figure 1: Motivating HotpotQA example. After review (2) detects missing evidence in draft (1), AFlow still executes its fixed `format` suffix and scores 0. Our TROVE instead applies `INSERT` to add `revise` (3), followed by `REPLACE` to substitute `stop` (4) for the obsolete suffix, preserving the useful prefix and raising the score to 1.

before the outcomes that matter become observable. For example, AFlow selects a dataset-level workflow (Zhang et al. 2025b); MaAS chooses a query-specific architecture before execution (Zhang et al. 2025a); and LAS adapts only within a predefined graph (Xiang et al. 2026). However, as execution unfolds, a queued continuation can become stale: review may demand revision, success may render verification redundant, or tool failure may invalidate downstream steps. Executing that continuation compounds errors and wastes computation.

Such runtime mismatch motivates the new requirement of adaptive route validation and editing: after each execution boundary, validate the pending route and edit it at the smallest sufficient scope. Unlike restarting or regenerating the entire future, this strategy preserves the executed prefix and artifacts, retains valid steps, repairs local mismatches, and replans only an invalid suffix. Figure 1 illustrates the idea on HotpotQA: `review` exposes missing evidence, but AFlow still executes the queued `format` step and fails. We instead insert `revise` and replace the obsolete suffix with `stop`, preserving the useful draft-review prefix while changing only what the new evidence invalidates.

Building on this insight, we propose Trace-grounded

Route Orchestration via Validation and Editing, **TROVE** for short. Offline, TROVE distills evaluated workflow search traces into a skill registry and a transition graph: stable fragments become executable composite skills, whereas outcome-dependent continuations remain explicit. Online, a planner proposes a short route, but the controller commits to only its first top-level skill. After observing outcome, TROVE retains the planned successor if it remains valid, inserts a local response if the downstream intent remains sound, or invokes an LLM to replace only the invalid suffix. TROVE thus separates planning from commitment: its proposal horizon spans multiple skills, while its commitment horizon remains one action.

We summarize the contributions of this work as follows:

- We identify a fundamental agent orchestration challenge: adapting a workflow when runtime outcomes invalidate what it planned to do next.
- We propose TROVE, which distills evaluated search traces into executable composite skills and an outcome-conditioned transition graph, then combines one-step commitment with different route revisions to preserve useful structure and artifacts.
- We evaluate TROVE on 6 code, question-answering, and mathematics benchmarks with 3 LLMs. Across 18 backbone-benchmark settings, TROVE achieves the best or tied-best task score in 15 and the shortest online time in 16. Relative to AFlow, TROVE improves task score in 16 settings (by up to 31.37%) and reduces time in 16 (by up to 86.7%).

2 Related Work

2.1 Workflow Construction Before Execution

Automated workflow construction spans *operation-*, *workflow-*, and *query-level* structure. At the operation level, GPTSwarm represents agent systems as computational graphs and optimizes node prompts and connectivity through task feedback (Zhuge et al. 2024). At the workflow and system level, AFlow searches executable code workflows, while ADAS programs complete agent systems from an archive of prior designs (Zhang et al. 2025b; Hu, Lu, and Clune 2025). AFlow is representative of this level: it uses execution feedback to optimize a dataset-level workflow that is subsequently reused at test time. Query-level methods instead construct or select an architecture for each instance, including MaAS, DyLAN, and FlowReasoner (Zhang et al. 2025a; Liu et al. 2023; Gao et al. 2025). MaAS, for example, samples a customized multi-agent architecture from an optimized agentic supernet before execution. Across these levels, the main structure is determined before instance-specific intermediate outcomes are observed. TROVE instead treats a proposed skill route as provisional and revises only its unexecuted continuation online.

2.2 Skill Abstraction and Experience Reuse

Skill-oriented methods differ in how reusable capability is obtained and represented. Classical temporal abstraction and SayCan assume predefined executable policies or skills, with

SayCan selecting actions according to both linguistic relevance and environmental feasibility (Sutton, Precup, and Singh 1999; Ahn et al. 2022). Code as Policies moves from selection to synthesis by composing perception and control APIs into executable programs with functions, conditions, and loops (Liang et al. 2023). Experience-based methods derive reuse from previous interactions: ExpeL extracts textual insights from trajectories, whereas Voyager stores acquired behaviors as an executable code-skill library (Zhao et al. 2024; Wang et al. 2023). Recent systems further organize and compose large skill collections through capability trees, executable DAGs, or evolving skill-relation graphs (Li et al. 2026a; Xia et al. 2026; Li et al. 2026b). TROVE similarly operationalizes experience, but derives its units specifically from evaluated workflow-search traces: existing operators become atomic skills, stable recurring fragments become composite skills, and outcome-dependent continuations remain explicit in the transition graph.

2.3 Execution-Time Planning and Route Adaptation

Execution-time controllers differ in how much future structure they preserve after feedback. ReAct generates only the next action after each observation (Yao et al. 2023). Plan-level methods such as AdaPlanner and DoReMi retain a high-level plan and refine or recover it when execution feedback violates its assumptions (Sun et al. 2023; Guo et al. 2023). State- and workflow-level controllers instead restrict adaptation to structured transition spaces: StateFlow routes execution through a state machine, LAS dynamically selects paths within a workflow DAG, and FlowSwitch decides when to continue or switch workflows according to dialogue state (Wu et al. 2024; Xiang et al. 2026; Chang et al. 2026). LAS is the closest representative, using intermediate outputs to selectively exit, verify, repair, or reroute within an existing workflow. TROVE instead edits the unresolved suffix of a task-specific provisional route: it retains a valid successor, inserts a local transition-supported response when possible, and replaces the suffix only when broader replanning is required. This preserves the executed prefix and still-valid route intent without restricting adaptation to a predefined DAG.

3 Design of TROVE

3.1 Problem Formulation and Overview

Problem Formulation. Let $\mathcal{S}$ be a registry of executable skills and $R = (s_1, \dots, s_k)$ a skill route proposed before its intermediate outcomes are observed. After executing s_t , the latest outcome may invalidate the pending continuation: the planned successor may lack a required artifact, a queued repair may become unnecessary, the route may be exhausted before the task is complete, or repeated actions may cease to make progress. We refer to this mismatch between the proposed continuation and the observed execution state as *continuation invalidation*.

Existing systems commonly address this problem at two extremes. A coarse-grained strategy abandons the current workflow, selects another complete workflow, and re-executes the task. Although this may replace an unsuitable

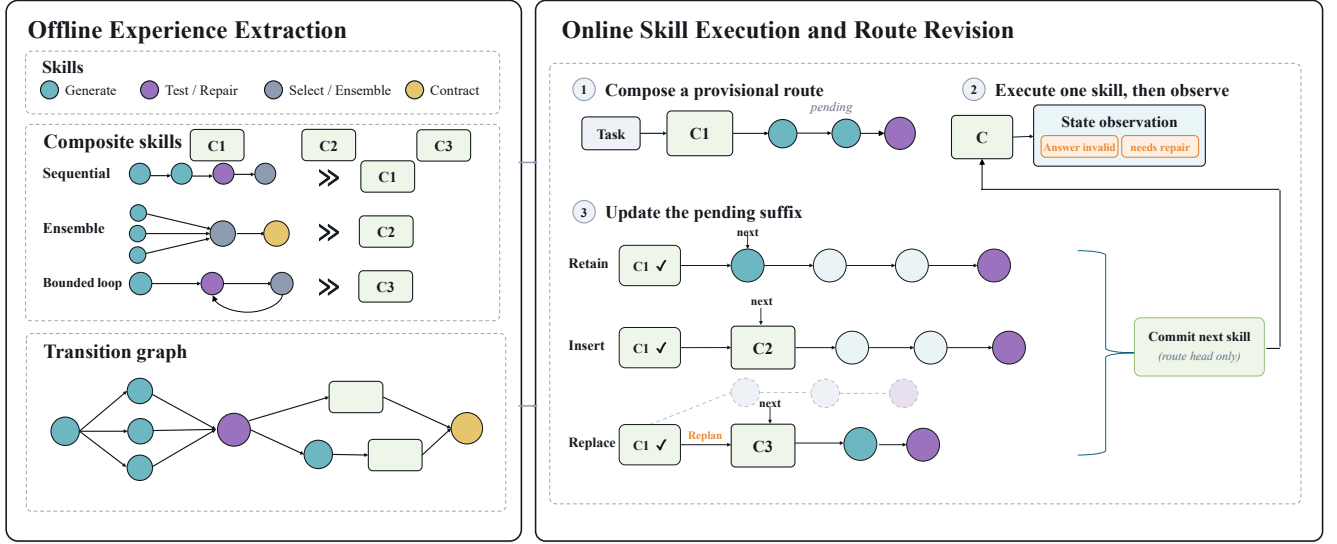


Figure 2: Offline experience extraction and online route revision in TROVE. Offline, standardized atomic skills are organized into stable composite skills and a transition graph (left). Online, a planner proposes a provisional route, but the controller executes only its head and observes the resulting state. The pending suffix is then retained, extended by inserting a locally supported skill, or replaced through replanning. Only the selected next skill is committed before the execution–observation loop repeats (right).

global structure, it also discards useful progress and repeats completed model calls, tool executions, and intermediate artifacts. At the other extreme, step-wise LLM replanning selects a new action or regenerates the remaining route after every node. This provides greater reactivity, but repeatedly incurs routing latency and token cost, introduces decision variance, and may overwrite portions of the original route that remain valid.

TROVE occupies the middle ground between these strategies. It treats a planner-generated route as provisional intent and commits to only one top-level skill at a time. After observing the outcome of the committed skill, TROVE retains the planned continuation when it remains valid, inserts a bounded local response when the original intent can still be preserved, and replaces only the unexecuted suffix when broader replanning is necessary.

Method Overview. TROVE separates offline experience construction from online route adaptation. Offline, AFlow operators are standardized as atomic skills, and evaluated one-node search traces are organized into two complementary forms of reusable experience: stable local fragments are encapsulated as composite skills, while outcome-dependent continuations are retained in a skill transition graph.

Online, a planner uses the resulting skill registry and transition graph to propose a short route. The controller executes only its first skill and, after observing the boundary outcome, decides whether to terminate, retain the queued successor, insert a local response, or invoke an LLM to replace the unexecuted suffix. Thus, the proposal horizon may cover multiple skills, while the commitment horizon remains one top-level action.

3.2 Offline Experience Extraction

TROVE starts from the operators used by AFlow. Since each operator is an atomic execution node in a workflow, we wrap it as an atomic skill with a standardized interface describing its required inputs, produced artifacts, execution status, and repeat constraints. These skills form the initial registry $\mathcal{S}_A$ and the primitive action space of offline workflow search.

AFlow searches workflows on the training partition. In each iteration, it modifies one workflow node by adding or substituting an operator and evaluates the resulting workflow. Let W_i and W_{i+1} denote two consecutive candidates. The corresponding changes are

$$\begin{aligned}\Delta_i^{\text{score}} &= \text{Score}(W_{i+1}) - \text{Score}(W_i), \\ \Delta_i^{\text{time}} &= \text{Time}(W_{i+1}) - \text{Time}(W_i).\end{aligned}\tag{1}$$

Together with the executed skill sequence and intermediate outcomes, these paired evaluations provide localized comparative evidence about how a structural modification affects task quality and execution cost. TROVE therefore reuses the search trajectory rather than retaining only the final workflow selected by AFlow.

Atomic and Composite Skill Abstraction. Atomic skills remain the primitive execution units. Using the operator sequences and one-node modifications recorded during AFlow search, TROVE identifies recurring local fragments in effective trajectories as composite candidates. A candidate is promoted to a composite skill when its internal ordering and data dependencies remain stable, it implements a coherent subgoal, exposes a consistent input–output contract, and can execute without consulting the pending outer route. If an intermediate outcome may change a later top-level decision,

the fragment remains decomposed so that the corresponding boundary is preserved.

Accepted fragments form the composite set $\mathcal{S}_C$, yielding

$$\mathcal{S} = \mathcal{S}_A \cup \mathcal{S}_C. \quad (2)$$

Atomic and composite skills expose the same top-level interface. Once invoked, a composite executes its child skills internally and returns one aggregate boundary outcome to the outer controller. Their occurrences are collapsed into composite nodes in the search traces, aligning offline experience with the top-level actions used by the online controller.

Composite abstraction serves a specific role in TROVE: it hides stable local decisions behind one execution boundary while leaving outcome-dependent continuations exposed to the online controller.

Skill Transition Graph. The same search traces also reveal relations that should not be hidden inside a composite. When the appropriate continuation depends on an observed outcome, TROVE retains the relation explicitly in a skill transition graph. After accepted fragments are mapped to composite nodes, a transition is represented as

$$\tau = (s, e, c, g), \quad s, g \in \mathcal{S}, \quad (3)$$

where s is the completed top-level skill, e is the observed boundary event, c summarizes the observable boundary features shared by the supporting trace fragments, and g is one supported immediate response skill.

An edge is derived from an evaluated local workflow modification rather than from skill adjacency alone. The corresponding score and time changes serve as offline comparative evidence for retaining the proposed response; they are not used as routing inputs during online execution.

At deployment, the graph serves two bounded roles. Before execution, the planner uses nominal continuation hints supported by task-visible context. After a skill completes, graph lookup additionally conditions on the observed boundary outcome and provides a next-skill candidate when the pending route becomes invalid. The graph does not construct a complete workflow or directly edit the remaining suffix. Whether the retrieved candidate should be inserted locally or used as a prior for LLM-based suffix replacement is decided by the online controller.

3.3 Online Skill Execution and Route Revision

Provisional Route and One-Step Execution. At deployment, a planner uses the task x , the offline-constructed skill registry $\mathcal{S}$, and the skill transition graph $\mathcal{G}$ to propose a short route

$$R_0 = \text{Plan}(x, \mathcal{S}, \mathcal{G}) = (s_1, \dots, s_k). \quad (4)$$

The graph provides bounded continuation hints rather than a complete path. Accordingly, the route is treated as provisional intent: TROVE commits to only its first top-level skill and keeps the remaining skills as an unexecuted suffix.

At iteration t , let $R_t = (s_t) \oplus Q_t$, where s_t is the committed skill, Q_t is the provisional suffix, and H_t is the preceding execution history. TROVE executes s_t , observes its boundary outcome o_t , and updates the history as $H_{t+1} = H_t \oplus (s_t, o_t)$. The outcome summarizes the execution status, available artifacts, and signals relevant to the next routing decision.

Observation-Guided Route Update. If the boundary outcome indicates that the task is complete, TROVE returns the current result and discards the pending suffix. Otherwise, it examines the queued successor. *Retain* is used when that successor remains executable under the latest outcome: the controller preserves the original suffix and continues without another LLM routing call.

If the successor is absent or no longer applicable, TROVE queries the transition graph using the completed skill and the observed boundary state. The graph returns one trace-supported response candidate when available and abstains otherwise. We denote the lookup result by $g_t \in \mathcal{S} \cup \{\perp\}$, where $g_t = \perp$ indicates graph abstention. The controller then revises the route through one of two operations.

Insert is used when g_t addresses the immediate mismatch—for example, through repair, testing, or verification—without changing the downstream objective encoded by the pending suffix. TROVE places the response before that suffix:

$$R_{t+1} = (g_t) \oplus Q_t. \quad (5)$$

After g_t completes, the original successor is checked again rather than executed automatically. When Q_t is empty, the same operation acts as a bounded route extension.

Replace is used when $g_t = \perp$, when the graph evidence is ambiguous, or when the observed state requires a broader change to the continuation. It is also used when a graph-supported response indicates a new direction that cannot preserve the downstream objective of the pending suffix. An LLM replanner receives the task, the latest outcome, the updated execution history, the pending suffix, the available skills, and g_t as a routing prior when it is available. It then generates a bounded replacement suffix:

$$\hat{Q}_t = \text{Replan}(x, o_t, H_{t+1}, Q_t, \mathcal{S}; g_t), \quad R_{t+1} = \hat{Q}_t. \quad (6)$$

When $g_t = \perp$, the replanner proceeds without a graph recommendation. Unlike workflow reselection and re-execution, replacement preserves the completed prefix and its artifacts and regenerates only the unexecuted portion.

Iterative Execution. TROVE repeatedly commits to the first skill of the updated route and applies the same outcome-guided update after each boundary. Execution terminates when a final result is produced or when the bounded step, repetition, or no-progress constraints are reached. If a budget or no-progress bound is reached before a valid final result is produced, the execution is marked incomplete and scored according to the benchmark evaluator.

4 Evaluation

4.1 Experimental Setup

Benchmarks and Data Partitions. We evaluate on six benchmarks spanning code generation, mathematical reasoning, and question answering: HumanEval (Chen et al. 2021), MBPP (Austin et al. 2021), MATH (Hendrycks et al. 2021), GSM8K (Cobbe et al. 2021), DROP (Dua et al. 2019), and HotpotQA (Yang et al. 2018). Each benchmark is divided approximately 1:4 into a training partition and a held-out test

Table 1: Combined task quality and online elapsed time on the held-out test partitions. Score is reported in percent and time in minutes. HumanEval and MBPP use pass@1, DROP and HotpotQA use mean token F1, and MATH and GSM8K use accuracy. Within each backbone and benchmark, **bold** and underlining independently mark the best and second-best value for each metric. “–” indicates that LAS does not publicly release a trained gate for that non-code benchmark.

Backbone	Method	Code				QA				Math			
		HumanEval		MBPP		DROP		HotpotQA		MATH		GSM8K	
		Score ↑	Time ↓	Score ↑	Time ↓	Score ↑	Time ↓	Score ↑	Time ↓	Score ↑	Time ↓	Score ↑	Time ↓
DeepSeek-V4-Flash	AFlow (Zhang et al. 2025b)	93.89	21.75	91.79	43.10	<u>88.50</u>	<u>15.56</u>	<u>78.58</u>	<u>25.41</u>	<u>74.07</u>	68.88	96.02	<u>57.73</u>
	MaAS (Zhang et al. 2025a)	87.02	<u>13.52</u>	81.82	35.13	81.64	22.98	77.37	26.60	68.11	131.83	<u>96.68</u>	108.23
	LAS (Xiang et al. 2026)	<u>94.66</u>	13.81	<u>92.08</u>	<u>22.02</u>	–	–	–	–	–	–	–	–
	Our TROVE	97.71	8.10	93.55	12.63	90.39	14.89	80.16	23.97	80.86	26.08	97.16	9.63
GPT-4o-mini	AFlow (Zhang et al. 2025b)	<u>93.13</u>	5.18	75.95	9.42	<u>65.61</u>	7.83	68.46	11.26	52.26	36.76	<u>92.23</u>	<u>37.79</u>
	MaAS (Zhang et al. 2025a)	95.42	8.19	74.49	20.68	34.55	13.13	57.90	11.73	51.44	122.04	92.80	149.02
	LAS (Xiang et al. 2026)	84.73	<u>2.94</u>	80.65	<u>6.01</u>	–	–	–	–	–	–	–	–
	Our TROVE	89.31	1.76	<u>78.59</u>	2.17	80.82	<u>8.06</u>	70.05	10.93	<u>52.06</u>	8.01	92.80	8.04
Qwen3-8B	AFlow (Zhang et al. 2025b)	<u>90.08</u>	60.53	51.91	116.58	<u>85.46</u>	<u>42.48</u>	<u>75.91</u>	44.05	8.44	273.35	<u>87.04</u>	<u>228.38</u>
	MaAS (Zhang et al. 2025a)	65.65	<u>43.11</u>	62.76	96.84	63.04	67.92	66.17	57.82	<u>12.55</u>	<u>235.70</u>	76.21	346.33
	LAS (Xiang et al. 2026)	89.31	44.08	<u>76.25</u>	<u>66.75</u>	–	–	–	–	–	–	–	–
	Our TROVE	92.37	24.36	83.28	41.84	85.85	37.23	76.03	<u>54.27</u>	36.42	36.41	92.80	44.18

partition. The resulting training/test sizes are 33/131 for HumanEval, 86/341 for MBPP, 119/486 for MATH, 264/1,055 for GSM8K, and 200/800 for both DROP and HotpotQA. Workflow search, controller configuration, trace collection, composite-skill construction, and transition-memory construction use only the training partition. All test-time structures are frozen before the corresponding test partition is evaluated.

Baselines. We compare TROVE with three representative workflow orchestration methods operating at different adaptation granularities. AFlow (Zhang et al. 2025b) searches a dataset-level workflow over operator nodes on the training set and executes the selected workflow unchanged at test time. It represents offline workflow optimization and evaluates whether outcome-aware route revision improves over replaying a strong fixed workflow. MaAS (Zhang et al. 2025a) samples a query-specific multi-agent architecture from an optimized agentic supernet before execution. It allows the workflow structure to vary across queries, but does not adjust it according to intermediate outcomes, providing a comparison with pre-execution adaptation. LAS (Xiang et al. 2026) dynamically selects the next node or path within a workflow DAG using intermediate outputs. As the closest online baseline, it compares dynamic next-hop scheduling with TROVE’s validation and retain–insert–replace revision of an interruptible skill route.

Together, these baselines cover dataset-level offline optimization, query-level pre-execution adaptation, and node-level online scheduling.

Metrics. We report task quality, online execution time, and online token usage. *Task score* is pass@1 for HumanEval and MBPP, accuracy for MATH and GSM8K, and mean token-level F1 for DROP and HotpotQA; all values are shown as percentages. *Time* is the elapsed wall-clock time required to evaluate the complete test partition under concurrency 10. It includes online planning, skill execution, route validation, and any runtime replanning, but excludes one-time offline workflow search, controller training, registry con-

struction, and trace distillation. *Tokens* are provider-reported input-plus-output totals over online planner, router, skill, and composite-internal model calls. Scores are compared only within the same benchmark.

Implementation Details. We evaluate AFlow, MaAS, LAS, and TROVE with DeepSeek-V4-Flash, GPT-4o-mini, and Qwen3-8B whenever supported by their released implementations. All test runs use a maximum concurrency of 10 and a 300-second per-sample timeout.

AFlow deploys workflows selected on the training partition. MaAS uses its frozen controller, while LAS uses the released gate checkpoints. All offline artifacts of TROVE are frozen before test evaluation.

The final TROVE registry contains 10 atomic skills and 41 composite skills, and the deployed transition graph contains 244 executable edges. The initial planner proposes at most four top-level skills, while online execution is limited to six top-level skill calls. No test result is used to select a workflow, controller, or checkpoint.

4.2 Main Results and Analysis

Overall Results. Table 1 compares all methods under the same execution backbone. Across the 18 backbone–benchmark blocks, TROVE obtains the best or tied-best task score in 15 and the shortest online time in 16. Against AFlow, TROVE improves score in 16 settings, with positive gains of 0.12–31.37 points on the corresponding task metric, and reduces time in 16 by 2.9–86.7%. Against MaAS, it improves or ties score in 17 settings, with positive gains of 0.48–46.27 points, and reduces time in all 18 by 6.1–94.6%. On the six LAS-supported code settings, it improves score in five by 1.47–7.03 points and reduces time in all six by 37.3–63.9%. The gains reflect different advantages over each baseline: avoiding stale suffix replay relative to AFlow, adapting after intermediate outcomes unavailable to MaAS, and reconstructing the pending route beyond LAS’s predefined DAG.

Across the six DeepSeek-V4-Flash test partitions, TROVE uses 13.45M online tokens versus AFlow’s 21.11M, a 36.3% reduction. The reduction is driven primarily by MBPP and

GSM8K, while DROP and HotpotQA require more tokens under TROVE.

Differences Across Tasks. Performance varies with both task difficulty and where that difficulty arises. On the harder MATH benchmark, TROVE gains 6.79 points over AFlow with DeepSeek-V4-Flash while reducing time by 62.1%; multi-step reasoning creates more opportunities for intermediate outcomes to invalidate later operations. GSM8K is near the performance ceiling, so score gains are smaller, but early completion still yields large time savings. HumanEval changes routes more frequently than MBPP, while code quality remains strongly backbone-dependent. The moderate gains on DROP and HotpotQA suggest that their difficulty lies more in evidence extraction and long-context processing than in unstable continuations. Overall, adaptation yields larger quality gains when task difficulty changes what should happen next, but mainly improves efficiency on high-scoring or evidence-heavy tasks.

Differences Across Backbones. Model capability changes whether adaptation yields quality gains or mainly efficiency. DeepSeek-V4-Flash shows the most consistent pattern, with TROVE improving both score and time on all six tasks, suggesting reliable use of composite execution and boundary feedback. GPT-4o-mini retains large time savings but shows mixed scores; strong baseline results on HumanEval and GSM8K leave less headroom and make the outcome more sensitive to routing choices. Qwen3-8B gains most on MBPP and MATH, where lower baseline scores leave more room for structured execution and selective revision.

Route-Adaptation Frequency. Figure 3 reports how often the realized route differs from its proposal. HumanEval changes 54.20% of routes, indicating substantial demand for runtime correction, whereas MBPP (5.87%), DROP (7.75%), and HotpotQA (2.88%) usually retain the initial route. MATH (98.97%) and GSM8K (100%) show the opposite pattern: an early composite answer often makes the remaining suffix unnecessary, so most edits reflect terminal pruning rather than failure recovery. Edit rate therefore represents corrective adaptation on HumanEval, early completion on mathematics, and route stability on MBPP and QA.

4.3 Ablation Study

We conduct ablation on HumanEval with Qwen3-8B. Each variant removes one source of offline structure or online adaptation while leaving the remaining controller unchanged. Relative to Full TROVE, *Without Offline* removes both composite skills and the transition graph and falls by 14.51 accuracy points while adding 9.26 minutes, showing the cost of relying on atomic skills and online planning alone. *Without Composite* retains the graph but removes composite execution; its similar loss of 13.74 points and 8.94-minute increase suggest that stable composite structure accounts for most of the offline benefit in this setting. *Without Insert* retains suffix replacement but removes local graph-supported repair, reducing accuracy by 9.93 points and adding 14.74 minutes as intermediate failures require broader replanning. *Without Replace* retains insertion but removes LLM suffix replan-

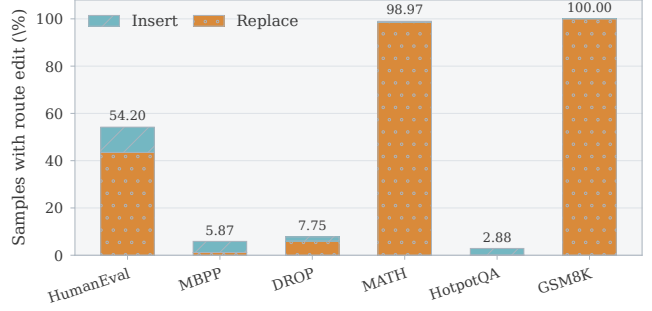


Figure 3: Sample-level incidence of TROVE’s online route edits with DeepSeek-V4-Flash. Bars stack INSERT and REPLACE; labels give the total percentage of samples whose realized route differs from its proposal. Terminal pruning counts as replacement, while timeout-incomplete paths and identity confirmations are excluded.

Table 2: Qwen3-8B HumanEval test ablation. Time is online test elapsed wall time.

Variant	Accuracy (%) ↑	Time (min) ↓
Full TROVE	92.37	24.36
Without Offline	77.86	33.62
Without Composite	78.63	33.30
Without Insert	82.44	39.10
Without Replace	90.84	40.15

ning; accuracy decreases by only 1.53 points, while time rises by 15.79 minutes. Thus, replacement appears less critical for correctness than for pruning or escaping unsuitable continuations efficiently.

5 Conclusion

In this work, we have formulated continuation invalidation as a distinct orchestration problem and identified adaptive validation and minimal editing of an unexecuted route as the corresponding test-time requirement. We further proposed a trace-grounded framework TROVE that distills evaluated workflow-search trajectories into executable composite skills and an outcome-conditioned transition graph, and combines them with a retain–insert–replace controller that preserves completed work. Experiments across six benchmarks and three LLM backbones show that selectively revising a provisional skill route improves both task quality and online efficiency over dataset-level optimization, query-level pre-execution adaptation, and graph-constrained online scheduling. More broadly, TROVE advances agent orchestration from static workflow execution toward adaptive, experience-grounded control that efficiently responds to runtime feedback while preserving useful progress.

References

Ahn, M.; et al. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. In *Conference on Robot Learning*.

- Austin, J.; Odena, A.; Nye, M.; Bosma, M.; Michalewski, H.; Dohan, D.; Jiang, E.; Cai, C.; Terry, M.; Le, Q.; and Sutton, C. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732*.
- Chang, W. Y.; Qiu, L.; Liu, Y.-H.; and Chen, Y.-N. 2026. FlowSwitch: A State-Aware Framework for Workflow Transitions in Adaptive Dialogue Agents. In *Proceedings of the 16th International Workshop on Spoken Dialogue System Technology*, 18–33. Association for Computational Linguistics.
- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. d. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.
- Dua, D.; Wang, Y.; Dasigi, P.; Stanovsky, G.; Singh, S.; and Gardner, M. 2019. DROP: A Reading Comprehension Benchmark Requiring Discrete Reasoning over Paragraphs. In *Proceedings of NAACL-HLT*, 2368–2378.
- Gao, H.; Liu, Y.; He, Y.; Dou, L.; Du, C.; Deng, Z.; Hooi, B.; Lin, M.; and Pang, T. 2025. FlowReasoner: Reinforcing Query-Level Meta-Agents. *arXiv preprint arXiv:2504.15257*.
- Guo, Y.; Wang, Y.-J.; Zha, L.; and Chen, J. 2023. DoReMi: Grounding Language Model by Detecting and Recovering from Plan-Execution Misalignment. *arXiv preprint arXiv:2307.00329*.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring Mathematical Problem Solving with the MATH Dataset. *arXiv preprint arXiv:2103.03874*.
- Hu, S.; Lu, C.; and Clune, J. 2025. Automated Design of Agentic Systems. In *The Thirteenth International Conference on Learning Representations*.
- Li, H.; Mu, C.; Chen, J.; Ren, S.; Cui, Z.; Zhang, Y.; Bai, L.; and Hu, S. 2026a. Organizing, Orchestrating, and Benchmarking Agent Skills at Ecosystem Scale. *arXiv preprint arXiv:2603.02176*.
- Li, X.; Li, M.; Bao, K.; Ma, Y.; Wang, W.; Liu, D.; and Feng, F. 2026b. SkillGraph: Skill-Augmented Reinforcement Learning for Agents via Evolving Skill Graphs. *arXiv preprint arXiv:2605.12039*.
- Liang, J.; Huang, W.; Xia, F.; Xu, P.; Hausman, K.; Ichter, B.; Florence, P.; and Zeng, A. 2023. Code as Policies: Language Model Programs for Embodied Control. In *2023 IEEE International Conference on Robotics and Automation*, 9493–9500.
- Liu, Z.; Zhang, Y.; Li, P.; Liu, Y.; and Yang, D. 2023. A Dynamic LLM-Powered Agent Network for Task-Oriented Agent Collaboration. *arXiv preprint arXiv:2310.02170*.
- Sun, H.; Zhuang, Y.; Kong, L.; Dai, B.; and Zhang, C. 2023. AdaPlanner: Adaptive Planning from Feedback with Language Models. *arXiv preprint arXiv:2305.16653*.
- Sutton, R. S.; Precup, D.; and Singh, S. 1999. Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning. *Artificial Intelligence*, 112(1–2): 181–211.
- Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*.
- Wu, Y.; Yue, T.; Zhang, S.; Wang, C.; and Wu, Q. 2024. StateFlow: Enhancing LLM Task-Solving through State-Driven Workflows. In *First Conference on Language Modeling*.
- Xia, T.; Hu, L.; Sun, Y.; Xu, M.; Xu, L.; Wang, S.; Xu, W.; and Jiang, J. 2026. GraSP: Graph-Structured Skill Compositions for LLM Agents. *arXiv preprint arXiv:2604.17870*.
- Xiang, D.; Chu, K.; Xu, W.; Zhang, W.; and Zhang, W. 2026. LLM-as-Scheduler: Agentic Workflow Dynamic Scheduling. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 12752–12763. Association for Computational Linguistics.
- Yang, Z.; Qi, P.; Zhang, S.; Bengio, Y.; Cohen, W.; Salakhutdinov, R.; and Manning, C. D. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of EMNLP*, 2369–2380.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.
- Zhang, G.; Niu, L.; Fang, J.; Wang, K.; Bai, L.; and Wang, X. 2025a. Multi-agent Architecture Search via Agentic Supernet. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 75834–75852. PMLR.
- Zhang, J.; Xiang, J.; Yu, Z.; Teng, F.; Chen, X.-H.; Chen, J.; Zhuge, M.; Cheng, X.; Hong, S.; Wang, J.; Zheng, B.; Liu, B.; Luo, Y.; and Wu, C. 2025b. AFlow: Automating Agentic Workflow Generation. In *The Thirteenth International Conference on Learning Representations*.
- Zhao, A.; Huang, D.; Xu, Q.; Lin, M.; Liu, Y.-J.; and Huang, G. 2024. ExpeL: LLM Agents Are Experiential Learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 19632–19642.
- Zhuge, M.; Wang, W.; Kirsch, L.; Faccio, F.; Khizbullin, D.; and Schmidhuber, J. 2024. GPTSwarm: Language Agents as Optimizable Graphs. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 62743–62767. PMLR.